

מבוא לאלגוריתמים

תרגיל בית 1 – חזרה על גרפים

1. א. תארו אלגוריתם המדפיס את היער המתקבל מהרצת אלגוריתם BFS
ב. תארו אלגוריתם המדפיס את היער המתקבל מהרצת אלגוריתם DFS

2. **הגרף המוחלף (transpose)** של גרף מכוון $G = (V, E)$ הוא הגרף $G^T = (V, E^T)$ כאשר $E^T = \{(v, u) \in V \times V \mid (u, v) \in E\}$. כלומר, G^T מתקבל מ- G על ידי היפוך כיווני כל הקשתות ב- G .

תארו אלגוריתמים יעילים לחישוב G^T בהינתן G , הן עבור ייצוג G על ידי מטריצת שכנויות והן על ידי ייצוג G על ידי הייצוג המעורב (ייצוג על ידי מערך של מצביעים לרשימות השכנים). נתחו את זמני הריצה של האלגוריתמים.

3. נתון גרף קשיר לא מכוון. לכל קשת (u, v) משויך משקל שלם $w(u, v)$ שערכו בין 1 לבין קבוע k . תארו אלגוריתם שזמן הריצה שלו הוא $O(|V| + k|E|)$ אשר בהינתן קודקוד s מוצא את המסלול בעל המשקל הכולל הקטן ביותר מ- s לכל קודקוד אחר בגרף. (שימו לב לנתח את זמן הריצה ולהראות שהוא אכן כפי הנדרש).

הבהרה: משקל של צומת משמש למעשה כאורך שלה

מבוא לאלגוריתמים

תרגיל בית 1 – חזרה על גרפים

1. א. תארו אלגוריתם המדפיס את היער המתקבל מהרצת אלגוריתם BFS.

תזכורת:

אלגוריתם BFS, כפי שנלמד, מקבל כקלט גרף וקודקוד מקור s ועובר רק על הקודקודים הנגישים מקודקוד המקור. לצורך הדפסת התשובה, מעבר להוספת ההדפסה, נצטרך לבצע שינויים כדי שהאלגוריתם יעבור על כל קודקודי הגרף וידפיס את כל היער המתקבל מהסריקה ולא רק את עץ הסריקה מקודקוד המקור.

מוטיבציה לתשובה:

נסתכל על אלגוריתם DFS שנלמד, ונסה להבין מה השינויים שנצטרך לבצע כדי להדפיס את כל היער ולא רק עץ בודד. אפשר לראות שאלגוריתם BFS כפי שנלמד, מזכיר את פונקציית העזר DFS-visit באלגוריתם DFS.

print_BFS_Forest(G)

```
1. for each vertex  $u \in V[G]$ :
2.   do color[u] ← WHITE
3.     d[u] ←  $\infty$ 
4.      $\pi[u]$  ← NIL
5. tree_number ← 0
6. for each vertex  $u \in V[G]$ 
7.   do if color[u] = WHITE
8.     then tree_number ← tree_number + 1
9.     print("tree number" + tree_number)
10.    BFS(G, u)
```

BFS(G, s)

```
1.  $Q \leftarrow \emptyset$ 
2. color[s] ← GRAY
3. print(v)
4. Enqueue(Q, s)
5. while  $Q \neq \emptyset$ 
6.   do  $u \leftarrow \text{Dequeue}(Q)$ 
7.     for each  $v \in \text{Adj}[u]$ 
8.       do if color[v] = WHITE
9.         then color[v] ← GRAY
10.        print(v)
11.        d[v] ← d[u] + 1
12.         $\pi[v]$  ← u
13.        Enqueue(Q, v)
14.    color[u] ← BLACK
```

הוכחת נכונות

נרצה להראות שהאלגוריתם מדפיס את היער שמתקבל מסריקת BFS.

אלגוריתם BFS, כפי שנלמד בכיתה, מקבל גרף וקודקוד מקור ומבקר בכל עץ הקודקודים הנגישים לקודקוד המקור.

כדי להדפיס את כל יער BFS, נצטרך לעבור על כל קודקודי הגרף ולא רק מקודקוד מקור נתון.

את אתחול הגרף נבצע לפני הרצת האלגוריתם הנלמד, בדומה ל DFS ואת האתחול נוציא מפונקציית BFS שלמדנו.

בגלל שכך או כך האתחול יתבצע לפני הלולאה בשורה 4 – הדבר לא ישפיע על נכונות אלגוריתם BFS.

במקום לסרוק עץ יחיד, לאחר כל סריקת עץ, במקום לעצור נמשיך לחפש קודקוד לבן (שלא חלק מעץ הסריקה הקודם, אחרת היה שחור) ונבצע סריקה נוספת. נסיים רק לאחר שנעבור על כל קודקודי הגרף.

בכל פעם שנפגוש קודקוד לראשונה (הרגע בו נצבע הקודקוד באפור), נדפיס אותו – כך נוודא שיער הסריקה יכיל את כל הקודקודים. לא נדפיס קודקוד פעמיים בגלל שברגע שנראה את הקודקוד באחת הסריקות, נשנה את הצבע שלו מצבע לבן והוא לא יטופל בשורה 8

סיבוכיות זמן

$\Theta(V+E)$ – אם באלגוריתם BFS שנלמד, סיבוכיות הזמן הייתה $O(V+E)$, כי נעבור רק על קודקודים וקשתות שנגישים לקודקוד המקור s , כאן נעבור על כל קשתות וקודקודי הגרף. בנוסף למעבר זה, הוספנו הדפסות בעלות $O(V)$ בסך הכל – $O(1)$ כל אחת, אשר ידפיסו את כל V הקודקודים.

מבוא לאלגוריתמים

תרגיל בית 1 – חזרה על גרפים

ב. תארו אלגוריתם המדפיס את היער המתקבל מהרצת אלגוריתם DFS

תזכורת:

אלגוריתם DFS, כפי שנלמד, עובר על כל קודקודי הגרף ומחלק את קודקודי הגרף לעצים. לצורך הדפסת התשובה, נצטרך רק להוסיף הדפסות.

DFS(G)

```
1.  for each vertex  $u \in V[G]$ 
2.    do color[u] ← WHITE
3.       $[[u] \leftarrow \text{NIL}$ 
4.  time ← 0
5.  tree_number ← 0
6.  for each vertex  $u \in V[G]$ 
7.    do if color[u] = WHITE
8.      then tree_number ← tree_number +
9.        print("tree number" + tree_number)
10.     DFS-VISIT(u)
```

DFS-VISIT(u)

```
1.  color[u] ← GRAY
2.  print(s)
3.  time ← time + 1
4.  d[u] ← time
5.  for each  $v \in \text{Adj}[u]$ 
6.    do if color[v] = WHITE
7.      then  $[[v] \leftarrow u$ 
8.          DFS-VISIT(v)
9.  color[u] ← BLACK
10. f[u] ← time ← time + 1
```

הוכחת נכונות

נרצה להראות שהאלגוריתם מדפיס את היער שמתקבל מסריקת DFS. כפי שנלמד, כל קריאה ל DFS-VISIT מהלולאה שבאלגוריתם DFS היא קריאה לסריקת עץ ביער הסריקה של אלגוריתם DFS. בתוך תת-האלגוריתם DFS-VISIT נוסיף הדפסה לאחר כל ביקור בקודקוד. נכונות ההדפסה נובעת מנכונות האלגוריתם שנלמד.

סיבוכיות זמן

$\Theta(V+E)$ – הוספנו הדפסות בעלות $O(V)$ בסך הכל – $O(1)$ כל אחת, אשר ידפיסו את כל V הקודקודים, כלומר, לא פגענו בעלות האלגוריתם המקורי

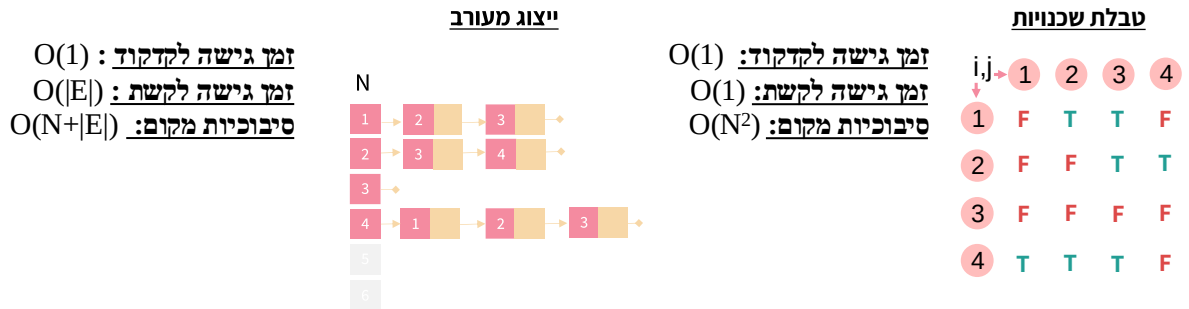
מבוא לאלגוריתמים

תרגיל בית 1 – חזרה על גרפים

2. **הגרף המוחלף (transpose)** של גרף מכוון $G = (V, E)$ הוא הגרף $G^T = (V, E^T)$, כאשר $E^T = \{(v, u) \in V \times V \mid (u, v) \in E\}$. כלומר, G^T מתקבל מ- G על ידי היפוך כיווני כל הקשתות ב- G .

תארו אלגוריתמים יעילים לחישוב G^T בהינתן G , הן עבור ייצוג G על ידי מטריצת שכנויות והן על ידי ייצוג G על ידי הייצוג המעורב (ייצוג על ידי מערך של מצביעים לרשימות השכנים).
נתחו את זמני הריצה של האלגוריתמים.

תזכורת – ייצוגי גרפים במחשב



N – גודל המערך הסטטי שיוקצה בייצוג המעורב ובייצוג טבלת השכנויות

פתרון – ייצוג מטריצת שכנויות:

מכיוון שאנחנו לא יודעים באילו מיקומים במטריצה המייצגת את גרף G יופיע 1 (שקיימת קשת), אין ברירה אלא לסרוק את כל המטריצה המייצגת את G .

ComputeTranspose(G)

1. $G^T \leftarrow \text{int}[\text{MAX}][\text{MAX}]$ // Create new Graph
2. **for** $i \leftarrow 1$ to N // init Graph to empty graph by put zeros in every cell $\Theta(N^2)$
3. **do for** $j \leftarrow 1$ to N
4. **do** $G^T[i, j] = 0$
5. **for** $i \leftarrow 1$ to N
6. **do for** $j \leftarrow 1$ to N
7. **do if** $G[i, j] = 1$ // there is edge in G between vertex i and vertex j
8. **then** $G^T[j, i] = 1$ // there is edge in G^T between vertex j and vertex i

נכונות

נרצה להראות שהאלגוריתם מייצר את הגרף המוחלף נכון.
ע"פ ההגדרה של גרף מוחלף – אם בגרף המקורי הייתה קשת בין קודקוד i לקודקוד j , בגרף המוחלף תהיה קשת בין קודקוד j לקודקוד i (הקשת ההפוכה)

משורות 7 ו-8 באלגוריתם, ניתן לראות שאם בגרף המקורי הייתה קשת בין i ל- j (שורה 7), בגרף המוחלף תהיה קשת בין j ל- i (שורה 8).

עלות זמן

$\Theta(N^2)$ – כאשר N הוא קבוע סטטי שהוגדר מראש לצורת יצירת המטריצה. במקרה הטוב יהיה קרוב ל- $|V|$. זמן אתחול הגרף החדש והזמן של הלולאה המקוננת בשורות 5-8.

עלות מקום

$\Theta(N^2)$ – עלות מקום הגרף החדש

מבוא לאלגוריתמים

תרגיל בית 1 – חזרה על גרפים

2. הגרף המוחלף (transpose) של גרף מכוון $G = (V, E)$ הוא הגרף $G^T = (V, E^T)$, כאשר

$$E^T = \{(v, u) \in V \times V \mid (u, v) \in E\}$$

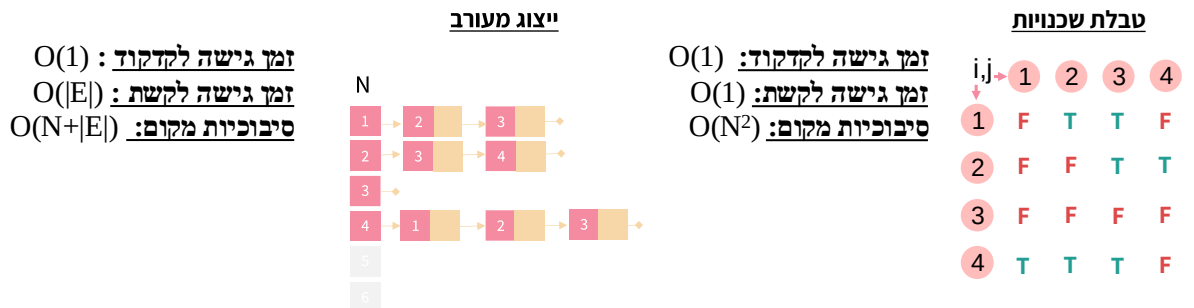
הקשתות ב- G .

תארו אלגוריתמים יעילים לחישוב G^T בהינתן G , הן עבור ייצוג G על ידי מטריצת שכנויות והן

על ידי ייצוג G על ידי הייצוג המעורב (ייצוג על ידי מערך של מצביעים לרשימות השכנים).

נתחו את זמני הריצה של האלגוריתמים.

תזכורת – ייצוגי גרפים במחשב



N – גודל המערך הסטטי שיוקצה בייצוג המעורב ובייצוג טבלת השכנויות

פתרון – ייצוג מטריצת שכנויות:

מכיוון שאנחנו לא יודעים באילו מיקומים במטריצה המייצגת את גרף G יופיע 1 (שקיימת קשת), אין ברירה אלא לסרוק את כל המטריצה המייצגת את G .

ComputeTranspose(G)

1. $G^T \leftarrow \text{Edge}^*[\text{MAX}]$ // Create new Graph
2. **for** $i \leftarrow 1$ to N // init Graph to empty graph by put Edge that will represent head of the linkedlist in every cell
3. **do** $G^T[i] = \text{new Edge}()$
4. **for** $u \leftarrow 0$ to N
5. $p \leftarrow G[u].\text{next}$
6. **while** $p \neq \text{NULL}$
7. $v \leftarrow p.\text{value}$ // edge (u, v) exist in G
8. $\text{temp} \leftarrow G^T[v].\text{next}$ // save previews V 's adj list
9. $G^T[v].\text{next} \leftarrow \text{new Edge}(u)$ // Create edge (v, u) in G^T as first in V 's Adj list
10. $G^T[v].\text{next.next} \leftarrow \text{temp}$ // Connect previews V 's adj list to new first Adj
11. $p \leftarrow p.\text{next}$

נכונות

נרצה להראות שהאלגוריתם מייצר את הגרף המוחלף נכון.

ע"פ ההגדרה של גרף מוחלף – אם בגרף המקורי הייתה קשת בין קודקוד i לקודקוד j , בגרף המוחלף תהיה קשת בין קודקוד j לקודקוד i (הקשת ההפוכה)

משורות 7 ו-9 באלגוריתם, ניתן לראות שאם בגרף המקורי הייתה קשת בין i ל- j (שורה 7), בגרף המוחלף תהיה קשת בין j ל- i (שורה 9). מהלולאה בשורה 4 והלולאה בשורה 6 ניתן לראות שנעבור על כל הקודקודים וכל רשימת שכני כל קודקוד (כל הקשתות)

עלות זמן

– כאשר N הוא קבוע סטטי שהוגדר מראש לצורת יצירת המטריצה. במקרה הטוב יהיה קרוב ל- $|V|$.

עלות מקום

– עלות מקום הגרף החדש $\Theta(N + |E|)$

מבוא לאלגוריתמים

תרגיל בית 1 – חזרה על גרפים

3. נתון גרף קשיר לא מכוון. לכל קשת (u, v) משויך משקל שלם $w(u, v)$ שערכו בין 1 לבין קבוע k . תארו אלגוריתם שזמן הריצה שלו הוא $O(|V| + k|E|)$ אשר בהינתן קודקוד s מוצא את המסלול בעל המשקל הכולל הקטן ביותר מ- s לכל קודקוד אחר בגרף. (שימו לב לנתח את זמן הריצה ולהראות שהוא אכן כפי הנדרש).

הבהרה: משקל של צומת משמש למעשה כאורך שלה

מוטיבציה לתשובה:

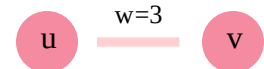
בתרגול ראינו שאחת מתכונותיו של אלגוריתם BFS היא מציאת המסלול הקצר ביותר מקודקוד המקור לקודקודים הנגישים לו. ננסה לחשוב על דרך להפוך משקל כל קשת לאורך מסלול, לדוגמה – בניית גרף חדש כך שמספר הקשתות בין קודקודי קשת (i, v) המקורית, יהיה כמשקל הקשת. באופן כזה, אפשר לחשוב על אלגוריתם BFS המקורי כאל גרף בו לכל קשת משקל 1.

פתרון:

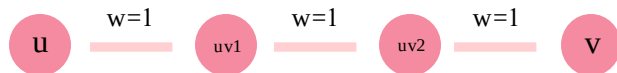
נבנה גרף G^w חדש באופן הבא: כל קשת (u, v) בגרף המקורי G בעלת משקל $w > 1$, נפצל את הקשת ל- w קשתות שונות, כלומר, נוסיף $k = w - 1$ קודקודים חדשים בין u ל- v . לאחר מכן נפעיל את אלגוריתם BFS על הגרף החדש וכתוצאה מכך המרחק d שיחושב עבור כל קודקוד יהיה שווה למשקל הכולל הקטן ביותר מקודקוד המקור בגרף המקורי G .

לדוגמה:

הקשת (u, v) בעלת משקל 3 בגרף המקורי:



תיראה כך בגרף G^w :



lightestPath(G, t)

```
1.  $G^w \leftarrow \text{new Graph}(V[G], \emptyset)$  //init new graph with vertex like original G and no edges
2. for  $(u, v) \in E[v]$ 
3.   do if  $w(u, v) > 1$ 
4.     then  $uv\_1 \leftarrow \text{new vertex}()$ 
5.         addVertex( $G^w$ ,  $uv\_1$ )
6.         addEdge( $G^w$ , new Edge( $u$ ,  $uv\_1$ ))
7.         for  $i \leftarrow 2$  to  $w(u, v) - 1$ 
8.           do  $uv\_i \leftarrow \text{new vertex}()$ 
9.               addVertex( $G^w$ ,  $uv\_i$ )
10.            addEdge( $G^w$ , new Edge( $u$ ,  $uv\_i$ ))
11.          $uv\_w-1 \leftarrow \text{new vertex}()$ 
12.         addVertex( $G^w$ ,  $uv\_w-1$ )
13.         addEdge( $G^w$ , new Edge( $uv\_w-1$ ,  $v$ ))
14.   else addEdge( $G^w$ ,  $(u, v)$ )
15. BFS( $G^w$ ,  $t$ )
```

נכונות

נרצה להראות שהאלגוריתם מחזיר את המסלול בעל המשקל הכולל הקטן ביותר. גרף G^w נוצר באופן בו מספר הקשתות בכל מסלול בין u, v , אם הייתה קיימת קשת כזו בגרף המקורי G , יהיה זהה למשקל הקשת המקורית. מתכונת אלגוריתם BFS המביא את המסלול הקצר ביותר בין קודקוד המקור לכל קודקוד, נוכל להבטיח שהאלגוריתם יעדיף מסלול בעל מספר קטן יותר של קשתות לכל קודקוד בגרף G^w , כלומר, משקל קטן יותר בגרף המקורי.

עלות זמן – במקרה הגרוע ביותר, המשקל של כל קשת בגרף המקורי הוא k ולכן בגרף G^w יהיו $k|E|$ קשתות ומכאן: $O(|V| + K|E|)$ כנדרש.